

Types d'analyse d'un algorithme

Fiche de synthèse — complexité algorithmique

1. Les grands axes d'analyse

Selon le moment de l'analyse

- **Analyse a priori** : étude théorique de l'algorithme avant toute exécution (complexité en temps/espace en fonction de la taille des données)
- **Analyse a posteriori** : mesure empirique des performances après exécution (chronométrage, profiling)

Selon le cas considéré (analyse a priori)

- **Meilleur cas** (best case) : configuration la plus favorable — borne inférieure
- **Pire cas** (worst case) : configuration la plus défavorable — analyse la plus utilisée car elle garantit une borne de performance
- **Cas moyen** (average case) : espérance du coût sur une distribution de données — plus réaliste mais plus difficile à établir

Selon ce qu'on mesure

- **Complexité temporelle** : nombre d'opérations élémentaires en fonction de la taille n
- **Complexité spatiale** : mémoire supplémentaire utilisée (hors données d'entrée)

Notations utilisées

- **$O(g)$** (grand O) : borne asymptotique supérieure — majoration du pire cas
- **$\Omega(g)$** (grand Oméga) : borne asymptotique inférieure
- **$\Theta(g)$** (grand Thêta) : encadrement exact (borne sup et inf du même ordre)

Types de complexité courants, par ordre croissant : $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$, $O(2^n)$, $O(n!)$

2. Zoom sur l'analyse a priori

C'est l'étude théorique de la complexité d'un algorithme, indépendamment de toute exécution réelle — donc indépendante du langage, du matériel, du compilateur.

Démarche générale

- 1 Identifier la taille des données n (longueur d'un tableau, nombre de nœuds, etc.)
- 2 Compter les opérations élémentaires en fonction de n (affectations, comparaisons, opérations arithmétiques...) — chacune coûte un temps constant
- 3 Exprimer le coût total $T(n)$ comme une fonction de n
- 4 Simplifier en ne gardant que l'ordre de grandeur dominant (notation O , Θ ...)

Exemple simple

```
def somme(tab):  
    s = 0 # 1 opération  
    for x in tab: # boucle n fois  
        s = s + x # 1 opération à chaque tour  
    return s # 1 opération
```

$T(n) = 1 + n \cdot 1 + 1 = n + 2$, donc $T(n) = O(n)$ — complexité linéaire.

Règles de calcul utiles

- **Séquence d'instructions** : on additionne les coûts, le terme dominant l'emporte. $O(n) + O(n^2) = O(n^2)$
- **Boucles imbriquées** : on multiplie les coûts. Deux boucles imbriquées sur $n \rightarrow O(n^2)$
- **Boucle qui divise à chaque étape** (ex : recherche dichotomique) $\rightarrow O(\log n)$
- **Appel récursif** : on établit une relation de récurrence puis on la résout (ex : $T(n) = 2T(n/2) + O(n)$ pour le tri fusion $\rightarrow O(n \log n)$, via le théorème maître)

Exemple à deux boucles imbriquées

```
def paires(tab):  
    for i in range(len(tab)): # n fois  
        for j in range(len(tab)): # n fois pour chaque i  
            print(tab[i], tab[j])
```

$T(n) = n \times n = O(n^2)$

Points d'attention

- On analyse généralement le pire cas, sauf mention contraire
- Les constantes et termes de bas degré disparaissent dans la notation O : $3n^2 + 5n + 7 \rightarrow O(n^2)$
- Pour du code avec des conditions (if), il faut parfois distinguer plusieurs scénarios (meilleur/pire cas)

3. Les trois cas : meilleur, pire, moyen

Pour un même algorithme, le nombre d'opérations peut varier selon la configuration précise des données en entrée, pas seulement selon leur taille n . D'où la nécessité de distinguer plusieurs scénarios.

Meilleur cas (best case)

- La configuration d'entrée la plus favorable, celle qui minimise le nombre d'opérations
- Donne une borne inférieure sur le temps d'exécution
- Peu utilisé en pratique car trop optimiste

Exemple : recherche séquentielle — meilleur cas : l'élément recherché est en première position $\rightarrow O(1)$

Pire cas (worst case)

- La configuration la plus défavorable, qui maximise le nombre d'opérations
- Donne une garantie : l'algorithme ne fera jamais pire que ça
- C'est l'analyse la plus utilisée, fiable pour des applications critiques (temps réel, systèmes embarqués...)

Exemple : recherche séquentielle — pire cas : l'élément est en dernière position, ou absent $\rightarrow O(n)$

Cas moyen (average case)

- L'espérance mathématique du coût, sur une distribution de probabilité des entrées (souvent supposée uniforme)
- Plus réaliste que le pire cas pour beaucoup d'usages, mais plus difficile à établir : il faut connaître ou supposer la distribution des données

Exemple : recherche séquentielle — en moyenne, l'élément est trouvé à la position $n/2 \rightarrow O(n)$ également, mais avec une constante plus petite que le pire cas

Exemple récapitulatif : tri rapide (quicksort)

Cas	Complexité	Explication
Meilleur	$O(n \log n)$	Le pivot partage le tableau en deux moitiés égales à chaque fois
Pire	$O(n^2)$	Le pivot est systématiquement le plus petit ou le plus grand élément (tableau déjà trié, mauvais choix de pivot)
Moyen	$O(n \log n)$	Sur des entrées aléatoires, les partitions déséquilibrées sont rares

C'est un bon exemple pédagogique : ça montre bien pourquoi le pire cas seul ne suffit pas à juger un algorithme — quicksort a un pire cas médiocre mais reste très utilisé grâce à son excellent comportement moyen.

4. Le modèle de référence : le modèle RAM

Le modèle RAM (Random Access Machine) est le modèle de référence standard pour l'analyse de complexité en algorithmique. C'est un modèle abstrait de calcul, indépendant de toute machine physique réelle.

Hypothèses du modèle RAM

- **Mémoire adressable directement** : accéder à n'importe quelle case mémoire se fait en temps constant $O(1)$, quel que soit l'endroit — d'où le nom « Random Access »
- **Opérations élémentaires à coût unitaire** : addition, comparaison, affectation, lecture/écriture mémoire... coûtent chacune $O(1)$
- **Taille des mots mémoire bornée** : les nombres manipulés tiennent dans un mot machine (word RAM)
- **Pas de parallélisme** : une seule opération exécutée à la fois (modèle séquentiel)

Pourquoi ce modèle ?

- Il correspond raisonnablement bien au fonctionnement réel d'un ordinateur (contrairement à la machine de Turing, plus adaptée à la calculabilité)
- Il permet de compter les opérations simplement et de comparer des algorithmes indépendamment du matériel ou du langage

Limites à connaître

- Il ignore la hiérarchie mémoire (cache, RAM, disque) — cf. modèles alternatifs (external memory model, cache-oblivious model)
- Il ignore le parallélisme (multi-cœurs, GPU)
- Pour des nombres arbitrairement grands, l'hypothèse « coût constant par opération » devient fausse → on utilise alors le modèle bit RAM

Pour l'essentiel d'un enseignement de classe prépa (tri, recherche, structures de données classiques), le modèle RAM standard suffit largement.